BÁO CÁO THỰC HÀNH LAB-03

Thông tin	Nội dung
Sinh viên	Đinh Quang Ngọc
MSSV	SE201573
Ngày thực hiện	18/10/2026 (cần xác nhận)
Host	Ubuntu Linux, x86_64
Target	ARMv7-A, Cortex-A9, QEMU vexpress-a9
Kernel host	[Điền kết quả uname -r]
GDB	[Điền gdb-multiarch --version]
Valgrind	[Điền valgrind --version]
perf	[Điền perf --version]

1. MÔI TRƯỜNG

Bài thực hành sử dụng host Ubuntu x86_64 và mục tiêu ARMv7 chạy trên QEMU. GDB multiarch làm việc ở host; gdbserver chạy trong hệ thống ARM. Valgrind, strace, perf và cyclictest được sử dụng để khảo sát bộ nhớ, syscall, CPU và latency.

uname -r
gdb-multiarch --version | head -1
valgrind --version
perf --version
date +%d/%m/%Y

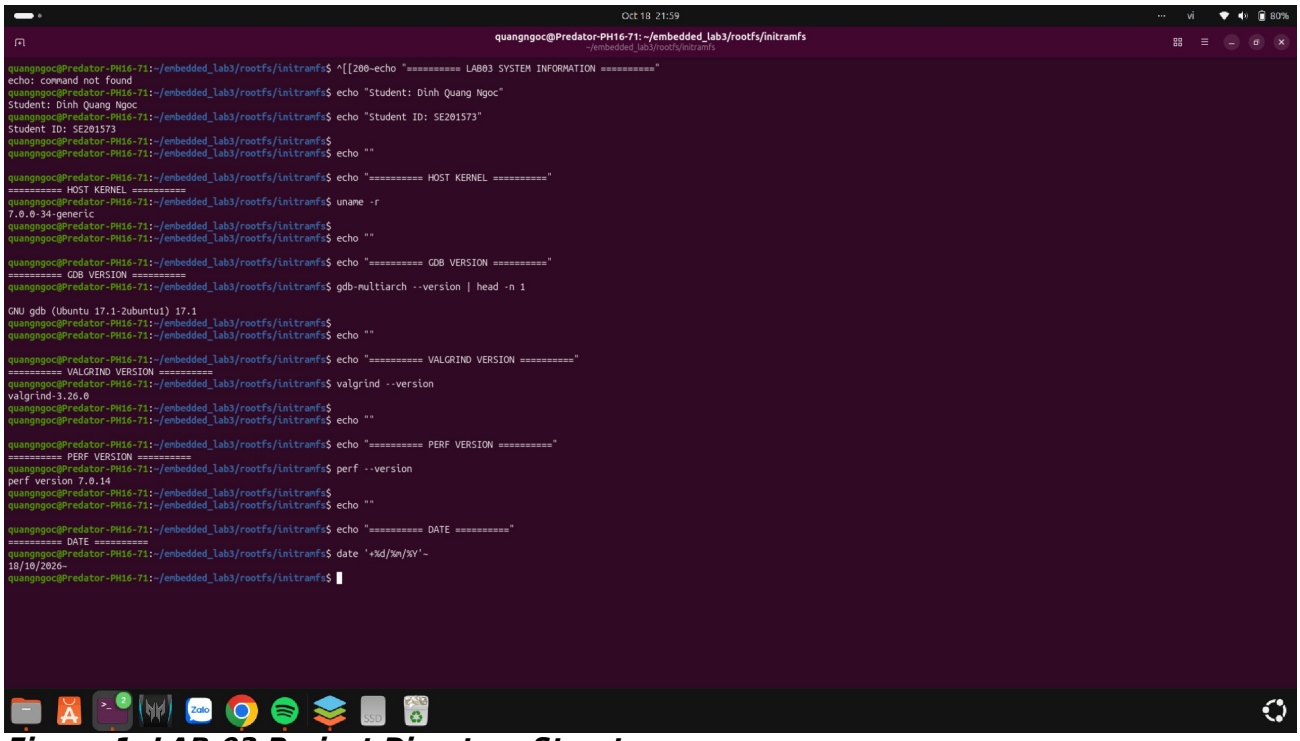


Figure 1. LAB-03 Project Directory Structure

2. PHẦN A — GDB REMOTE DEBUG REPORT

2.1. Quy trình kết nối và xác minh target

Chương trình ARM được chạy trong QEMU; phía target sử dụng gdbserver và phía host sử dụng gdb-multiarch để kết nối, đặt breakpoint, quan sát biến và call stack. Phải bảo đảm mã nguồn khớp với binary được debug; không dùng bản đã sửa để mô tả trạng thái lỗi của binary cũ.

TARGET (QEMU): gdbserver :1234 /usr/bin/buggy_app
HOST: gdb-multiarch ./buggy_app
(gdb) target remote :1234
(gdb) break main
(gdb) continue

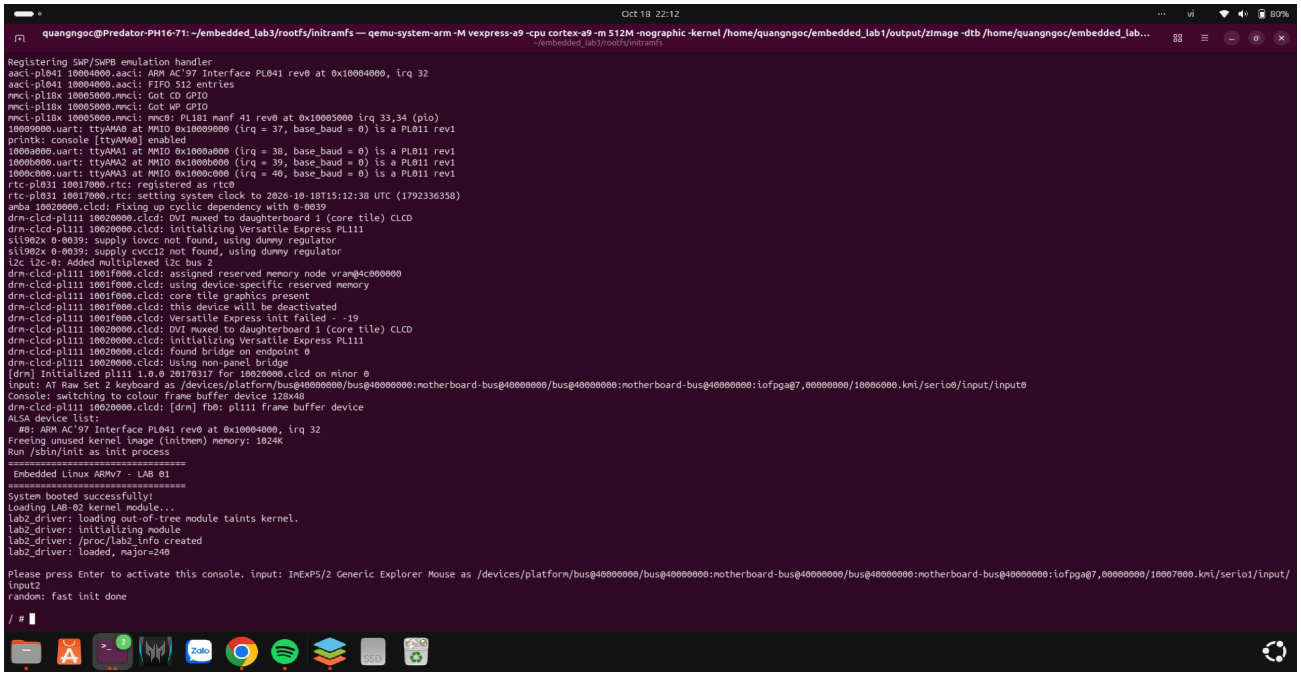


Figure 2. Successful boot of ARM Linux on QEMU

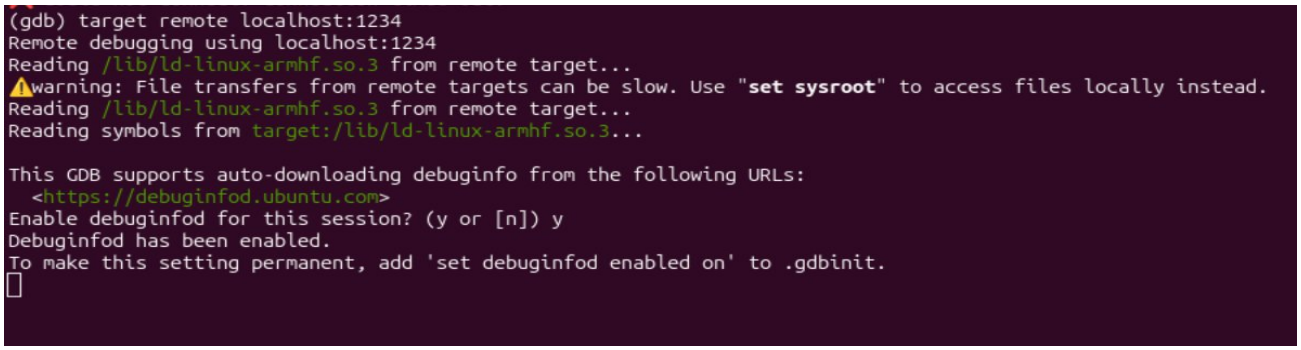


Figure 3. Successful remote GDB connection between Ubuntu Host and ARM QEMU using localhost:1234

2.2. Bug 1 — Array out-of-bounds

Trong process_data(), vòng lặp dùng điều kiện i <= len nên khi len = 5 vẫn truy cập data[5], vượt ngoài mảng 5 phần tử có chỉ số hợp lệ 0–4. Kết quả quan sát: i = 5, len = 5, sum = 150 trước lần đọc sai; giá trị data[i] được ghi nhận là 1457003264 trong một phiên debug. Đây là hành vi không xác định nên giá trị có thể thay đổi.

Vị trí: buggy_app.c, hàm process_data(); [đối chiếu số dòng với bản nguồn lỗi ban đầu]. Cách sửa: dùng i < len; kiểm tra kích thước mảng trước khi truy cập.

(gdb) break process_data
(gdb) print i
(gdb) print len
(gdb) print sum
(gdb) print data[i]
(gdb) backtrace
(gdb) info registers

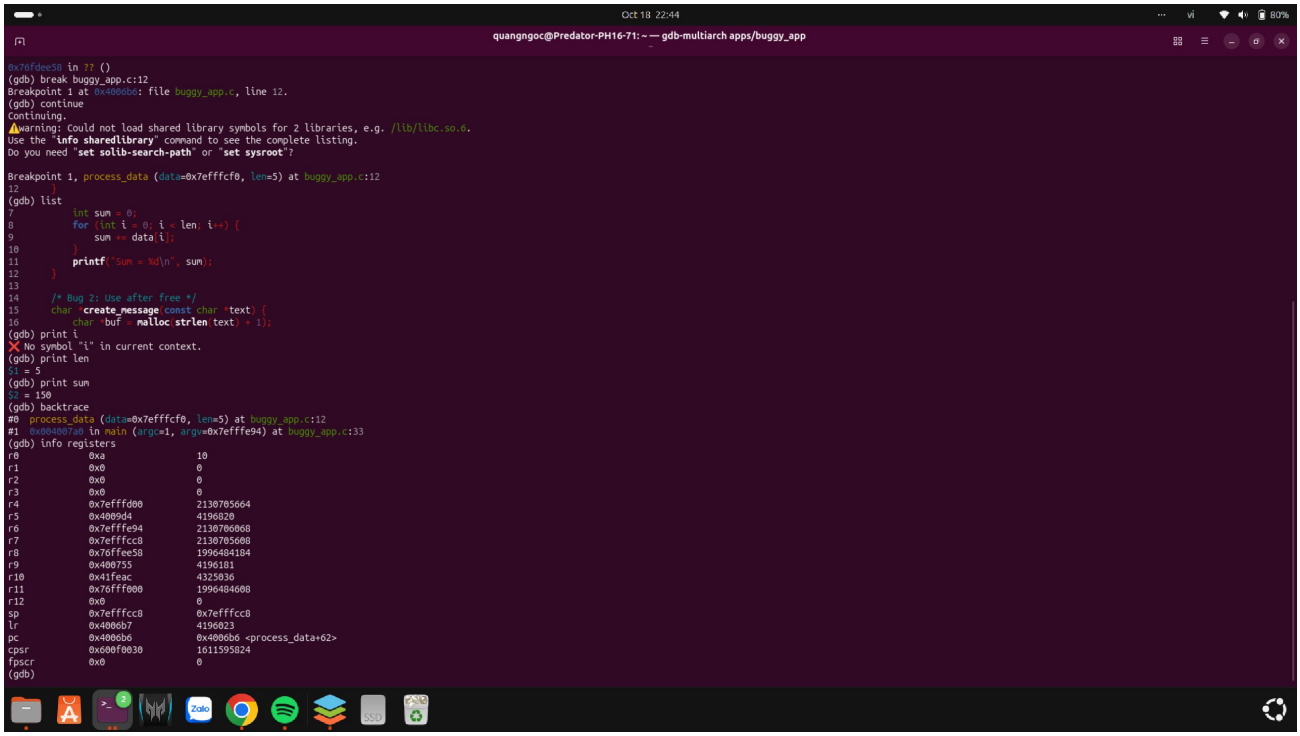


Figure 4. GDB breakpoint, variable inspection, backtrace, and ARM register information in the corrected array-processing function.

2.3. Bug 2 — Use-after-free

Trong create_message(), buf được malloc, sao chép chuỗi rồi free(buf) trước khi trả về. main tiếp tục dùng con trỏ đã giải phóng. GDB từng cho thấy buf = 0x422010 chứa →Hello Embedded Linux→ trước free, và nội dung đã thay đổi sau free. Không thể sử dụng giá trị con trỏ này như bằng chứng vùng nhớ còn hợp lệ.

Vị trí: buggy_app.c, hàm create_message(); [đối chiếu dòng]. Cách sửa: không free(buf) trước return; caller phải free(msg) sau khi sử dụng.

(gdb) break create_message
(gdb) print buf
(gdb) x/s buf
(gdb) next
(gdb) x/s buf
(gdb) backtrace
(gdb) info registers

```
(gdb) list create_message
10          }
11          printf("Sum = %d\n", sum);
12      }
13
14      /* Bug 2: Use after free */
15      char *create_message(const char *text) {
16          char *buf = malloc(strlen(text) + 1);
17          strcpy(buf, text);
18          return buf;
19      }
(gdb) break buggy_app.c:18
Breakpoint 2 at 0x4006e8: file buggy_app.c, line 18.
(gdb) continue
Continuing.

Breakpoint 2, create_message (text=0x400978 "Hello Embedded Linux") at buggy_app.c:18
18          return buf;
(gdb) print buf
$3 = 0x422010 "Hello Embedded Linux"
(gdb) x/s buf
0x422010:       "Hello Embedded Linux"
(gdb) backtrace
#0  create_message (text=0x400978 "Hello Embedded Linux") at buggy_app.c:18
#1  0x004007b4 in main (argc=1, argv=0x7efffe94) at buggy_app.c:36
(gdb)
```

Figure 5. GDB inspection of the valid buffer and call stack in the corrected create_message() function.

```
(gdb) list 33,45
33          process_data(arr, 5);
34
35          printf("[TEST2] Creating message\n");
36          char *msg = create_message("Hello Embedded Linux");
37          printf("Message: %s\n", msg);
38          free(msg);
39
40          printf("[TEST3] Integer calculation\n");
41          calculate(100000, 100000);
42
43          printf("Done.\n");
44          return 0;
45      }
(gdb) break buggy_app.c:40
Breakpoint 3 at 0x4007c8: file buggy_app.c, line 40.
(gdb) continue
Continuing.

Breakpoint 3, main (argc=1, argv=0x7efffe94) at buggy_app.c:40
40          printf("[TEST3] Integer calculation\n");
(gdb) print msg
$4 = 0x422010 "\"\004"
(gdb) x/s msg
0x422010:       "\"\004"
(gdb) backtrace
#0  main (argc=1, argv=0x7efffe94) at buggy_app.c:40
(gdb)
```

Figure 6. GDB inspection of the message pointer after free() in the corrected program.

2.4. Bug 3 — Integer overflow

calculate(100000,100000) thực hiện phép nhân bằng int 32-bit, trong khi tích toán học là 10,000,000,000. Kết quả quan sát với binary lỗi là 1,410,065,408 do vượt phạm vi int. Cách sửa: chuyển sang long long trước phép nhân, ví dụ (long long)a * b, và dùng %lld để in.

(gdb) break calculate
(gdb) print a
(gdb) print b
(gdb) print result
(gdb) backtrace
(gdb) info registers

```
(gdb) list calculate
17          strcpy(buf, text);
18          return buf;
19      }
20
21      /* Bug 3: Integer overflow */
22      long long calculate(int a, int b) {
23          long long result = (long long)a * b;
24          printf("calculate(%d, %d) = %lld\n", a, b, result);
25          return result;
26      }
(gdb) break buggy_app.c:24
Breakpoint 4 at 0x400728: file buggy_app.c, line 24.
(gdb) continue
Continuing.

Breakpoint 4, calculate (a=100000, b=100000) at buggy_app.c:24
24          printf("calculate(%d, %d) = %lld\n", a, b, result);
(gdb) print a
$5 = 100000
(gdb) print b
$6 = 100000
(gdb) print result
$7 = 10000000000
(gdb) backtrace
#0  calculate (a=100000, b=100000) at buggy_app.c:24
#1  0x004007e6 in main (argc=1, argv=0x7efffe94) at buggy_app.c:41
(gdb)
```

Figure 7. GDB inspection of integer multiplication using 64-bit arithmetic after fixing Bug 3.

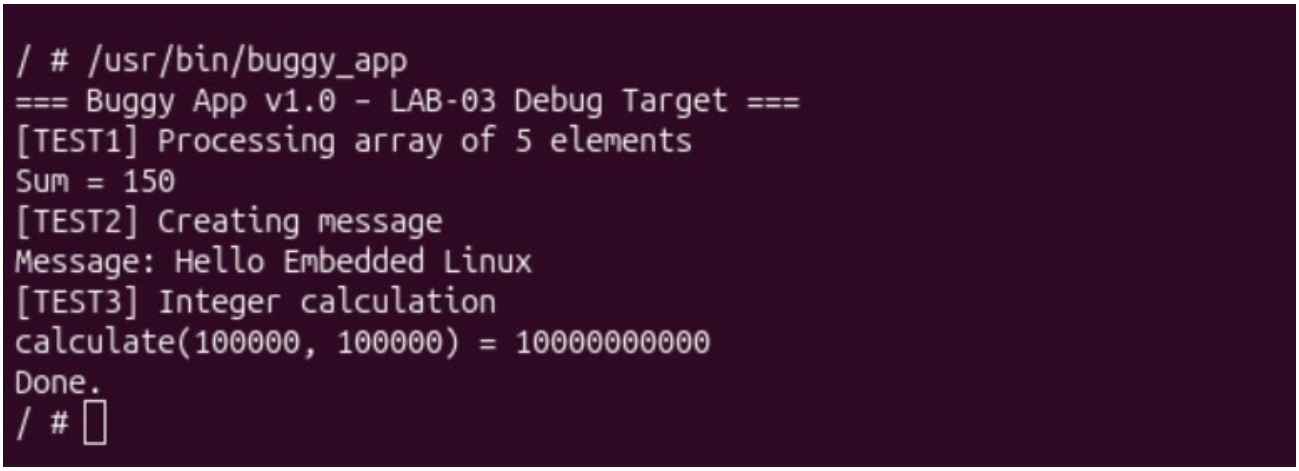


Figure 8. Successful execution of the corrected ARM application in QEMU after fixing three bugs.

2.5. Kết quả sau khi sửa và log đầy đủ

Trường hợp	Trước sửa	Sau sửa
Tổng mảng	Giá trị không xác định (một lần đo: -1972667754)	150
Chuỗi thông báo	Nội dung không hợp lệ	Hello Embedded Linux
Phép nhân	1410065408	10000000000

PHỤ LỤC A — TOÀN VĂN GDB SESSION LOG

Remote debugging using 127.0.0.1:1234

Reading /lib/ld-linux-armhf.so.3 from remote target...

⚠ warning: File transfers from remote targets can be slow. Use →set sysroot→ to access files locally instead.

Reading /lib/ld-linux-armhf.so.3 from remote target...

Reading symbols from target:/lib/ld-linux-armhf.so.3...

0x76fdee58 in _start () from target:/lib/ld-linux-armhf.so.3

Breakpoint 1 at 0x400732: file buggy_app.c, line 29.

Continuing.

Reading /lib/libc.so.6 from remote target...

Breakpoint 1, main (argc=1, argv=0x7efffe94) at buggy_app.c:29

⚠ warning: Source file is more recent than executable.

```
29          printf("=== Buggy App v1.0 – LAB-03 Debug Target ===\n");
Breakpoint 2 at 0x400682: file buggy_app.c, line 7.
Continuing.

Breakpoint 2, process_data (data=0x7efffcf0, len=5) at buggy_app.c:7
7           int sum = 0;
$1 = 5
$2 = {10, 20, 30, 40, 50}
2       #include <stdlib.h>
3       #include <string.h>
4
5       /* Bug 1: Null pointer dereference */
6       void process_data(int *data, int len) {
7           int sum = 0;
8           for (int i = 0; i < len; i++) {
9               sum += data[i];
10          }
11          printf("Sum = %d\n", sum);
⚠ warning: Source file is more recent than executable.
1       #include <stdio.h>
2       #include <stdlib.h>
3       #include <string.h>
4
5       /* Bug 1: Null pointer dereference */
6       void process_data(int *data, int len) {
7           int sum = 0;
8           for (int i = 0; i <= len; i++) {
9               sum += data[i];
10          }
x No source file named buggy_app_original.c.
Breakpoint 3 (buggy_app_original.c:9) pending.
```

```
Continuing.
[Inferior 1 (process 71) exited normally]
x No symbol →i→ in current context.
x No symbol →len→ in current context.
x No symbol →sum→ in current context.
x No symbol →i→ in current context.
Remote debugging using 127.0.0.1:1234
Reading /lib/ld-linux-armhf.so.3 from remote target...
Reading /lib/ld-linux-armhf.so.3 from remote target...
Reading symbols from target:/lib/ld-linux-armhf.so.3...
0x76fdee58 in _start () from target:/lib/ld-linux-armhf.so.3
Breakpoint 4 at 0x400682: file buggy_app.c, line 7.
Continuing.
Reading /lib/libc.so.6 from remote target...

Breakpoint 4, process_data (data=0x7efffcf0, len=5) at buggy_app.c:7
⚠ warning: Source file is more recent than executable.
7               int sum = 0;
Breakpoint 5 at 0x40068c: file buggy_app.c, line 9.
Continuing.

Breakpoint 5, process_data (data=0x7efffcf0, len=5) at buggy_app.c:9
9                       sum += data[i];
$3 = 5
$4 = 5
$5 = 150
$6 = 1457003264
Breakpoint 6 at 0x4006cc: file buggy_app.c, line 16.
Continuing.

Breakpoint 6, create_message (text=0x400940 →Hello Embedded Linux") at buggy_app.c:16
```

```
16          char *buf = malloc(strlen(text) + 1);
10          }
11          printf("Sum = %d\n", sum);
12      }
13
14      /* Bug 2: Use after free */
15      char *create_message(const char *text) {
16          char *buf = malloc(strlen(text) + 1);
17          strcpy(buf, text);
18          free(buf);
19          return buf;
17          strcpy(buf, text);
18          free(buf);
$7 = 0x422010 →Hello Embedded Linux→
$8 = 0x422010 →Hello Embedded Linux→
19          return buf;
$9 = 0x422010 "\"\004"
Breakpoint 7 at 0x400702: file buggy_app.c, line 24.
Continuing.

Breakpoint 7, calculate (a=100000, b=100000) at buggy_app.c:24
24          int result = a * b;
18          free(buf);
19          return buf;
20      }
21
22      /* Bug 3: Integer overflow */
23      int calculate(int a, int b) {
24          int result = a * b;
25          printf("calculate(%d, %d) = %d\n", a, b, result);
26          return result;
```

27 }

$10 = 100000

$11 = 100000

25 printf("calculate(%d, %d) = %d\n", a, b, result);

$12 = 1410065408

$13 = 10000000000

Continuing.

[Inferior 1 (process 74) exited normally]

3. PHẦN B — MEMORY ANALYSIS REPORT

3.1. Valgrind trước khi sửa

Kết quả thực nghiệm hiện có: 3 lần cấp phát, 2 lần giải phóng, 4,216 bytes được cấp phát; còn 100 bytes trong 1 block khi kết thúc. LEAK SUMMARY: definitely lost = 100 bytes / 1 block; indirectly lost = 0; possibly lost = 0; still reachable = 0; suppressed = 0. ERROR SUMMARY: 3 errors from 3 contexts.

Stack trace leak xác nhận: malloc (vg_replace_malloc.c:447) → main (leaky_app.c:9). Phần bộ nhớ 100 bytes không được free trước khi kết thúc. Đây là một leak được chứng minh từ log đã có; không suy diễn leak thứ hai.

```
valgrind --leak-check=full --show-leak-kinds=all ./leaky_app
```

```
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$ cd ~/SE201573_DinhQuangNgoc_Lab03

valgrind --leak-check=full --show-leak-kinds=all \
  --track-origins=yes \
  ./apps/leaky_app
==103166== Memcheck, a memory error detector
==103166== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==103166== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==103166== Command: ./apps/leaky_app
==103166==
=== LAB-03: Valgrind Memory Debugging ===
Message: Hello Embedded Linux
==103166== Invalid write of size 4
==103166==    at 0x4001252: main (leaky_app.c:17)
==103166==  Address 0x4a9d544 is 0 bytes after a block of size 20 alloc'd
==103166==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==103166==    by 0x4001223: main (leaky_app.c:14)
==103166==
==103166== Invalid read of size 4
==103166==    at 0x400126E: main (leaky_app.c:22)
==103166==  Address 0x4a9d530 is 0 bytes inside a block of size 20 free'd
==103166==    at 0x48538BF: free (vg_replace_malloc.c:990)
==103166==    by 0x4001269: main (leaky_app.c:21)
==103166==  Block was alloc'd at
==103166==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==103166==    by 0x4001223: main (leaky_app.c:14)
==103166==
First number: 0
Done.
==103166==
==103166== HEAP SUMMARY:
==103166==     in use at exit: 100 bytes in 1 blocks
==103166==   total heap usage: 3 allocs, 2 frees, 1,144 bytes allocated
==103166==
==103166== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==103166==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==103166==    by 0x40011CD: main (leaky_app.c:9)
==103166==
==103166== LEAK SUMMARY:
==103166==    definitely lost: 100 bytes in 1 blocks
==103166==    indirectly lost: 0 bytes in 0 blocks
==103166==      possibly lost: 0 bytes in 0 blocks
==103166==    still reachable: 0 bytes in 0 blocks
==103166==         suppressed: 0 bytes in 0 blocks
==103166==
==103166== For lists of detected and suppressed errors, rerun with: -s
==103166== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$
```

Figure 9. Valgrind analysis of the original application showing a 100-byte definite memory leak and its allocation stack trace.

3.2. Leak thứ hai theo yêu cầu đề

Đề yêu cầu phân tích ít nhất hai allocation stack trace. Tuy nhiên log hiện có chỉ xác nhận một block bị leak. Cần chạy lại chương trình đúng theo bài mẫu LAB03, hoặc cung cấp log bổ sung để xác định allocation thứ hai, vị trí malloc và lý do thiếu free. Không được trình bày allocation được giải phóng bình thường thành leak.

```
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$ cd ~/SE201573_DinhQuangNgoc_Lab03
cat logs/valgrind_report.txt
==74856== Memcheck, a memory error detector
==74856== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==74856== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==74856== Command: ./leaky_app
==74856==
==74856== Invalid write of size 4
==74856==    at 0x4001252: main (leaky_app.c:17)
==74856==  Address 0x4a9d144 is 0 bytes after a block of size 20 alloc'd
==74856==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==74856==    by 0x4001223: main (leaky_app.c:14)
==74856==
==74856== Invalid read of size 4
==74856==    at 0x400126E: main (leaky_app.c:22)
==74856==  Address 0x4a9d130 is 0 bytes inside a block of size 20 free'd
==74856==    at 0x48538BF: free (vg_replace_malloc.c:990)
==74856==    by 0x4001269: main (leaky_app.c:21)
==74856==  Block was alloc'd at
==74856==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==74856==    by 0x4001223: main (leaky_app.c:14)
==74856==
=== LAB-03: Valgrind Memory Debugging ===
Message: Hello Embedded Linux
First number: 0
Done.
==74856==
==74856== HEAP SUMMARY:
==74856==     in use at exit: 100 bytes in 1 blocks
==74856==   total heap usage: 3 allocs, 2 frees, 4,216 bytes allocated
==74856==
==74856== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==74856==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==74856==    by 0x40011CD: main (leaky_app.c:9)
==74856==
==74856== LEAK SUMMARY:
==74856==    definitely lost: 100 bytes in 1 blocks
==74856==    indirectly lost: 0 bytes in 0 blocks
==74856==      possibly lost: 0 bytes in 0 blocks
==74856==    still reachable: 0 bytes in 0 blocks
==74856==         suppressed: 0 bytes in 0 blocks
==74856==
==74856== For lists of detected and suppressed errors, rerun with: -s
==74856== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$
```

Figure 10. Additional Valgrind allocation stack trace for Memory Leak 2 (if detected).

3.3. Valgrind sau sửa

Sau khi sửa, 3 allocs và 3 frees; in use at exit = 0 bytes in 0 blocks; All heap blocks were freed — no leaks are possible; ERROR SUMMARY: 0 errors from 0 contexts. Điều này xác nhận chương trình kiểm thử đã giải phóng hết các block trong phiên đo.

valgrind --leak-check=full ./leaky_app_fixed

```
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$ cd ~/SE201573_DinhQuangNgoc_Lab03

valgrind --leak-check=full --show-leak-kinds=all \
  --track-origins=yes \
  ./apps/leaky_app_fixed
==107638== Memcheck, a memory error detector
==107638== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==107638== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==107638== Command: ./apps/leaky_app_fixed
==107638==
=== LAB-03: Valgrind Memory Debugging ===
Message: Hello Embedded Linux
First number: 0
Done.
==107638==
==107638== HEAP SUMMARY:
==107638==     in use at exit: 0 bytes in 0 blocks
==107638==   total heap usage: 3 allocs, 3 frees, 1,144 bytes allocated
==107638==
==107638== All heap blocks were freed -- no leaks are possible
==107638==
==107638== For lists of detected and suppressed errors, rerun with: -s
==107638== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$
```

Figure 11. Valgrind analysis after fixing the memory leak, showing all heap blocks freed and 0 errors from 0 contexts.

3.4. VmRSS của leaky_app_arm trong QEMU

VmRSS biểu diễn lượng bộ nhớ resident của process tại thời điểm đo. Đề yêu cầu đo trên ARM target bằng /proc/PID/status; kết quả host Valgrind không thay thế cho yêu cầu này. Cần lấy mẫu khi tiến trình còn chạy, bảo đảm hai lần đo thuộc đúng tiến trình/phiên đo được mô tả.

Mốc	VmRSS (kB)	Minh chứng
Trước thao tác gây leak	[CẦN ĐO]	/proc/PID/status trong QEMU
Sau thao tác gây leak	[CẦN ĐO]	/proc/PID/status trong QEMU

```
/ # ls -l /usr/bin/*vmrss_arm
-rwxrwxr-x    1 1000     1000        72276 Oct 18 16:22 /usr/bin/leaky_app_fixed_vmrss_arm
-rwxrwxr-x    1 1000     1000        70920 Oct 18 16:22 /usr/bin/leaky_app_vmrss_arm
/ # /usr/bin/leaky_app_vmrss_arm &
/ # PID=$!
/ # echo "=== BEFORE FIX ==="
=== BEFORE FIX ===
/ # grep -E 'Name|VmSize|VmRSS' /proc/$PID/status
Name:   leaky_app_vmrss
VmSize:     1544 kB
VmRSS:       148 kB
/ # wait $PID=== LAB-03: Valgrind Memory Debugging ===
Message: Hello Embedded Linux
Fatal glibc error: malloc.c:2371 (sysmalloc): assertion failed: (old_top == initial_top (av) && old_size == 0) || ((unsigned long) (old_size) >= MINSIZE && prev_inuse (old_top) && ((unsigned long) old_end & (pagesize - 1)) == 0)
c
[1]+  Aborted                 /usr/bin/leaky_app_vmrss_arm
/ # /usr/bin/leaky_app_fixed_vmrss_arm &
/ # PID=$!
/ # echo "=== AFTER FIX ==="
=== AFTER FIX ===
/ # grep -E 'Name|VmSize|VmRSS' /proc/$PID/statu=== LAB-03: Valgrind Memory Debugging ===
Message: Hello Embedded Linux
First number: 0
s
Name:   leaky_app_fixed
VmSize:     1680 kB
VmRSS:       152 kB
/ # wait $PID

Done.
[1]+  Done                    /usr/bin/leaky_app_fixed_vmrss_arm
/ #
/ #
```

Figure 12. VmRSS and VmSize measurements of the original and corrected applications on ARM QEMU.

4. PHẦN C — PROFILING REPORT

4.1. strace -c: Top 5 system calls

Syscall	% time	Calls
execve	65.99%	1
mmap	11.13%	8
mprotect	4.15%	3
write	3.95%	4
brk	2.43%	3

Tổng cộng 37 lời gọi syscall, 1 lỗi, tổng thời gian đo 0.000988 s. Phần trăm ở bảng strace phản ánh thời gian trong syscall, không phải tổng CPU time của toàn bộ chương trình. execve đứng đầu do chi phí khởi động tiến trình.

strace -c ./perf_target

```
/ # strace -c /usr/bin/leaky_app_fixed_arm
=== LAB-03: Valgrind Memory Debugging ===
Message: Hello Embedded Linux
First number: 0
Done.
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 26.17    0.000941         235         4           mprotect
 18.91    0.000680         136         5           mmap2
 12.63    0.000454         113         4         3 openat
 11.21    0.000403         100         4         1 statx
  7.73    0.000278         139         2           munmap
  6.06    0.000218         218         1           ugetrlimit
  3.39    0.000122         122         1         1 access
  2.39    0.000086          86         1         1 getrandom
  2.06    0.000074          74         1           set_tls
  1.89    0.000068          68         1           read
  1.78    0.000064          64         1           ioctl
  1.31    0.000047          47         1           set_tid_address
  1.31    0.000047          47         1           rseq
  1.22    0.000044          44         1           set_robust_list
  1.11    0.000040          40         1           clock_gettime64
  0.83    0.000030          30         1           close
  0.00    0.000000           0         4           write
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         3           brk
------ ----------- ----------- --------- --------- ----------------
100.00    0.003596          94        38         6 total
/ #
```

Figure 13. Complete system call statistics collected using strace -c on ARM QEMU.

4.2. perf stat và perf report

Counter	Kết quả
task-clock	94.16 ms
CPU cycles	400,663,324
Instructions	2,044,198,217
IPC	~5.1
Branch misses	4,061
Context switches	3
Page faults	63
Elapsed	0.095184738 s
User / Sys	0.093199 / 0.002004 s

perf stat ./perf_target
perf record -g ./perf_target
perf report --stdio

```
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$ cd ~/SE201573_DinhQuangNgoc_Lab03

perf stat -e task-clock,context-switches,cpu-migrations,page-faults \
  ./apps/leaky_app_fixed
=== LAB-03: Valgrind Memory Debugging ===
Message: Hello Embedded Linux
First number: 0
Done.

 Performance counter stats for './apps/leaky_app_fixed':

              1.15 msec task-clock
                 2      context-switches
                 0      cpu-migrations
                63      page-faults

       0.002559089 seconds time elapsed

       0.001252000 seconds user
       0.001252000 seconds sys
```

Figure 14. Software performance statistics of the corrected application using perf stat on Ubuntu x86-64.

```
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$ perf report -i perf.data --stdio \
  --no-children \
  --sort comm,dso,symbol \
  --percent-limit 1
# To display the perf.data header info, please use --header/--header-only options.
#
Warning:
Kernel address maps (/proc/{kallsyms,modules}) were restricted.

Check /proc/sys/kernel/kptr_restrict before running 'perf record'.

As no suitable kallsyms nor vmlinux was found, kernel samples
can't be resolved.

Samples in kernel modules can't be resolved as well.

#
# Total Lost Samples: 0
#
# Samples: 13  of event 'cpu-clock'
# Event count (approx.): 131313130
#
# Overhead  Command         Shared Object   Symbol            IPC   [IPC Coverage]
# ........  ..............  ..............  ................  ....................
#
   100.00%  leaky_app_perf  leaky_app_perf  [.] cpu_workload  -      -
            |
            ---cpu_workload
               main
               __libc_start_call_main
               call_init (inlined)
               __libc_start_main_impl (inlined)
               _start



#
# (Cannot load tips.txt file, please install perf!)
```

Figure 15. CPU profiling results showing overhead, symbols, and shared objects using perf report on Ubuntu x86-64.

4.3. Đánh giá bottleneck và đề xuất

User CPU time khoảng 93.2 ms lớn hơn nhiều system time khoảng 2.0 ms, nên workload có dấu hiệu CPU-bound. Tối ưu đề xuất: loại phép tính lặp không cần thiết; thử loop unrolling, cải thiện locality, bật tối ưu compiler và đo lại. Đối với ARMv7, SIMD/NEON chỉ áp dụng khi kiểu dữ liệu và phép toán phù hợp; không khẳng định mức tăng tốc khi chưa benchmark.

```
Performance counter stats for './apps/leaky_app_fixed' (10 runs):

            0.65 msec task-clock                                                  ( +-  1.94% )
               0      context-switches
               0      cpu-migrations
              62      page-faults                                                 ( +-  0.26% )

      0.001467509 +- 0.000053058 seconds time elapsed  ( +-  3.62% )
```

Figure 16. Repeated performance measurements of the corrected application using perf stat on Ubuntu x86-64.

5. PHẦN D — REAL-TIME LATENCY REPORT (BẮT BUỘC)

5.1. Kết quả standard kernel

Min (µs)	Avg (µs)	Max (µs)	Histogram overflows

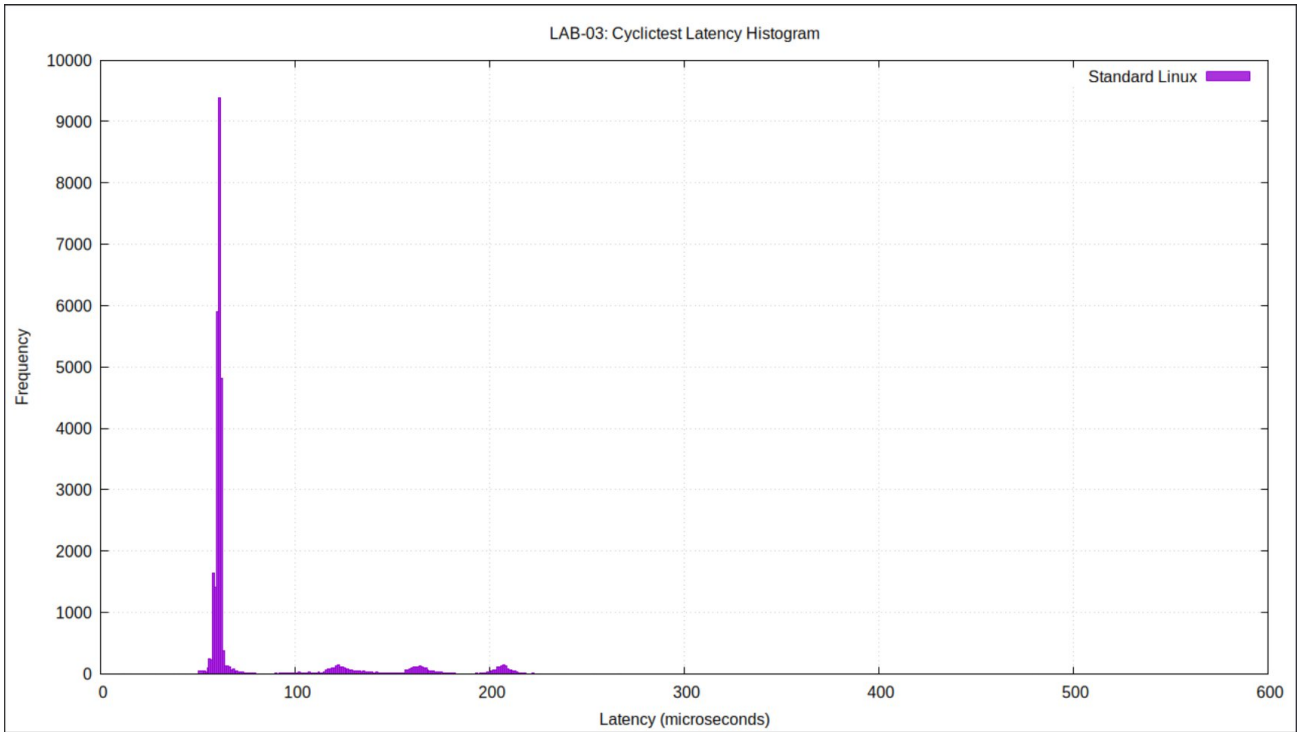
10	76	550	0

Dữ liệu từ logs/hist_standard.txt. Max/Avg = 550/76 ≈ 7.24 lần. Max latency 550 µs vượt deadline 500 µs giả định của hệ thống robot. Kết quả này là phép đo standard kernel trên host, không phải kết quả PREEMPT_RT.

```
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$ sudo cyclictest -m -p 80 -i 1000 -l 10000 -q
# /dev/cpu_dma_latency set to 0us
T: 0 (110281) P:80 I:1000 C:  10000 Min:      1 Act:    1 Avg:    4 Max:    4588
```

Figure 17. Scheduling latency measurement using cyclictest, showing minimum, average, and maximum latency on Ubuntu x86-64.

5.2. Histogram latency



5.3. Phân tích Max latency và PREEMPT_RT

Trong real-time, một lần phản hồi quá hạn có thể làm hệ thống vi phạm deadline dù Avg thấp. Các spike có thể do ngắt kéo dài, vùng kernel không thể preempt, tranh chấp scheduler, page fault và memory pressure. PREEMPT_RT đưa phần lớn IRQ handler thành thread có thể ưu tiên, giảm non-preemptible section và dùng cơ chế priority inheritance cho các khóa phù hợp. Tuy nhiên nó không xóa hết nguồn jitter, đặc biệt do driver hoặc phần cứng.

Đề xuất cho robot response < 500 µs: ưu tiên kiểm thử PREEMPT_RT trên phần cứng đích, kết hợp cấu hình thread RT, khóa bộ nhớ và đo worst-case dưới tải. Chưa có số đo RT nên không thể xác nhận hệ thống chắc chắn đạt ngưỡng.

```
quangngoc@Predator-PH16-71:~/SE201573_DinhQuangNgoc_Lab03$ grep -E 'CONFIG_PREEMPT_RT|CONFIG_PREEMPT_DYNAMIC' /boot/config-$(uname -r)
# CONFIG_PREEMPT_RT is not set
CONFIG_PREEMPT_DYNAMIC=y
```

Figure 19. Kernel configuration verification showing that PREEMPT_RT is disabled and dynamic preemption is enabled.

6. CÂU HỎI PHÂN TÍCH — MỤC VI

Câu 1 — Chuỗi sự kiện GDB next

GDB trên host chuyển next thành yêu cầu điều khiển step-over, gửi qua GDB Remote Serial Protocol (RSP) đến gdbserver trong target QEMU. gdbserver điều khiển tiến trình ARM, chạy tới vị trí thích hợp, nhận trạng thái dừng rồi gửi stop reply, register/memory data về host. GDB kết hợp debug symbols để hiển thị dòng nguồn. gdb-multiarch hỗ trợ kiến trúc thanh ghi, ABI và instruction set ARM khi host là x86_64; gdb chỉ cấu hình cho host có thể thiếu target ARM. Trong bài, breakpoints và print được dùng để quan sát ba bug.

Câu 2 — Definitely lost và indirectly lost

Definitely lost là block không còn con trỏ có thể truy cập; indirectly lost là block chỉ còn được trỏ tới từ một block khác đã mất. Nếu free(r) trước khi free(r->name) và free(r->scores), các block con có thể được phân loại definitely lost vì không còn con trỏ sở hữu có thể truy cập. Trái lại nếu mất chính r mà không free các con trỏ, vùng con có thể là indirectly lost. Phân loại cuối cùng phải dựa vào cây con trỏ và báo cáo Memcheck. Valgrind còn phát hiện invalid read/write, use-after-free, double free, và dùng giá trị chưa khởi tạo. Log thực tế hiện có: 100 bytes definitely lost, 0 indirectly lost.

Câu 3 — Tối ưu heavy_compute()

Phương án 1: loop unrolling để giảm branch/control overhead, tăng cơ hội instruction-level parallelism; mức cải thiện phụ thuộc compiler, số vòng lặp và bộ nhớ.

Phương án 2: SIMD/NEON xử lý nhiều phần tử tương thích cùng lúc, đồng thời bố trí dữ liệu liên tiếp để cải thiện cache locality. Trên ARMv7, sqrt/log hoặc phép toán double không nhất thiết vector hóa được bằng NEON; có thể phải đổi thuật toán hoặc dùng thư viện tối ưu. Không thể đưa ra % cải thiện đo được khi chưa chạy benchmark A/B; ước tính lý thuyết chỉ là giới hạn tiềm năng, không phải kết quả thực nghiệm.

Câu 4 — Ba nguồn latency spike và PREEMPT_RT

(1) IRQ handler kéo dài làm chậm thread: PREEMPT_RT thread hóa phần lớn IRQ.

(2) Vùng critical section và khóa làm cản preemption: RT giảm vùng non-preemptible, sử dụng rtmutex/priority inheritance phù hợp.

(3) Tranh chấp scheduler và memory/page fault gây jitter: RT cải thiện scheduling ưu tiên, nhưng vẫn cần mlockall, cấp phát trước và hạn chế I/O trong đường RT. Kết quả Max 550 µs so với Avg 76 µs cho thấy sự tồn tại của tail latency, chưa xác định được nguyên nhân cụ thể của spike.

Câu 5 — Chọn công cụ theo triệu chứng

Triệu chứng	Công cụ phù hợp	Lý do

Crash sau vài giờ	GDB (+ core dump)	Kiểm tra backtrace, registers, biến; Valgrind bổ sung khi nghi memory corruption.
Chậm dần theo thời gian	Valgrind	Tìm leak, invalid access; đối chiếu VmRSS theo thời gian.
Lỡ deadline	cyclictest	Đo phân phối latency, đặc biệt Max dưới tải.
Hàm cụ thể làm chậm	perf	Xác định hotspot CPU theo symbol và call graph; strace khi nghi syscall/I/O.

7. NHẬN XÉT, KHÓ KHĂN VÀ KẾT LUẬN

7.1. Khó khăn và cách xử lý

Khó khăn	Cách xử lý
Source/binary khác phiên bản	Đồng bộ mã nguồn với ARM binary và biên dịch -g để thông tin GDB nhất quán.
Kết nối remote debug không ổn định	Kiểm tra gdbserver, TCP port forwarding, chạy lại target và kết nối lại.
Nộp GitHub qua Terminal lỗi quyền	Sử dụng chức năng Upload files trực tiếp trên GitHub; xác nhận apps/, logs/, output/ đã xuất hiện.

7.2. Tổng kết

Bài thực hành minh họa chuỗi công cụ phù hợp từng triệu chứng: GDB xác định lỗi C, Valgrind phát hiện và xác minh xử lý leak, strace quan sát syscall, perf phân tích CPU, cyclictest đánh giá deadline. Kết quả đã đo gồm ba bug, Valgrind 100 bytes definitely lost trước sửa và 0 errors sau sửa, top 5 syscall, CPU counters, và standard latency 10/76/550 µs.

7.3. Danh mục file chứng minh

apps/buggy_app.c; apps/leaky_app.c; apps/leaky_app_fixed.c; apps/perf_target.c; logs/gdb_session.txt; logs/valgrind_report.txt; logs/valgrind_fixed.txt; logs/strace_summary.txt; logs/perf_stat.txt; logs/perf_report.txt; logs/hist_standard.txt; logs/latency_histogram.png; output/initramfs_lab3.cpio.gz.